

CI/CD em monorepo

Por que o pipeline que roda tudo é o primeiro gargalo de um projeto com dois serviços.

EQUIPE

Gilmar Silva · Julia Ribeiro
Ruan Pablo · Samuel Vanoni

GCC267 · 2026/2
Turma 14A

AGORA, NO SEU CELULAR

Entre no nosso repo e abra um pull request

1. Aponte a câmera para o QR ao lado
2. No GitHub, toque em **Add file** → **Create new file**
3. Nome do arquivo: **participantes/seunome.json**
4. Copie o modelo abaixo, troque seus dados
5. **Commit changes** → **Propose changes** → **Create pull request**

```
{ "nome": "Seu Nome",  
  "equipe": "sua equipe",  
  "tema": "T0X" }
```



ou digite:
[github.com/SamuVanoni/
GCC267-cicd-monorepo](https://github.com/SamuVanoni/GCC267-cicd-monorepo)

ENQUANTO VOCÊS MANDAM

Os próximos 30 minutos

MIN	QUEM	O QUE ACONTECE
00–05	Samuel	Vocês abrem os PRs
05–12	Julia	O pipeline julga — e a gente lê o log junto
12–20	Ruan	Por que isso quebra num monorepo
20–28	Gilmar	O conserto, o deploy e uma pegadinha

Nada aqui é slide com código de mentira. O que vocês mandarem agora é o que vai estar na tela no minuto 20.

O QUE É CI/CD, EM UMA FRASE

O pipeline é quem diz não.

Não é um robô que ajuda. É um porteiro que barra. E todo mundo que trabalha em equipe precisa de um, porque ninguém confia na palavra “testei aqui e funcionou”.

As duas letras, sem enrolação

CI

Integração contínua

Toda vez que alguém abre um PR, o robô baixa o código, instala, compila e roda os testes.

Entrega: um **check verde** ou um **X vermelho**.

CD

Entrega contínua

Se o check ficou verde e o código entrou na main, ele vai pro ar sozinho. Ninguém sobe nada à mão.

Entrega: a versão nova no ar.

push → **install** → **build** → **test** → merge → **deploy**
 isso é CI

isso é CD

O QUE ESTÁ RODANDO AGORA

O pipeline que todo mundo escreve no primeiro dia

Dez linhas, funciona, e é honesto. O problema é que ele não faz ideia do que mudou — então refaz tudo, do zero, toda vez.

sem cache — baixa as dependências de novo
sem filtro — testa os 3 pacotes sempre

```
on: pull_request

jobs:
  tudo:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test -w api
      - run: npm test -w mural
      - run: npm test -w shared
```

● AO VIVO — TROCAR PARA O NAVEGADOR

A parede

Os PRs de vocês, julgados um por um. Olhem quantos ficaram **verdes** e quantos tomaram **vermelho**.

`github.com/<org>/t05-demo/pulls`

Como ler um log de CI

1. Clique no **X vermelho** → **Details**
2. Ache o **step vermelho**. Os de cima dele passaram
3. Vá para o **fim** do log
4. Procure a primeira linha **Error:** — a de cima dela diz o arquivo

Você não lê o log.
Você lê o fim do log.

```
added 603 packages in 25s
> mural@1.0.0 test
> node validar.js
```

```
reading participantes/...
  joao.json ..... ok
  maria.json ..... ok
```

```
  ruan.json ..... FALHOU
```

```
Error: JSON invalido
  linha 4: virgula sobrando
  em participantes/ruan.json
```

```
Process exited with code 1
```

O PROBLEMA DO MONOREPO

Mudei um arquivo. Rodou tudo.

Vocês mudaram um **.json** com o próprio nome. O robô rodou os testes da API, do mural e da biblioteca compartilhada. Três vezes o trabalho, zero vezes o motivo.

O QUE TODAS AS NOVE EQUIPES VÃO TER

Monorepo: dois serviços, um repositório

```
hortalicas-feiralivre/  
  apps/  
    api/ ← serviço A  
    mural/ ← serviço B  
  packages/  
    shared/ ← os dois usam  
  .github/workflows/
```

O que ele resolve

Um PR muda o contrato e os dois lados dele juntos.
Um clone, um Compose, uma versão da verdade.

O que ele cobra

O pipeline não sabe mais o que mudou. Sem ensinar, ele trata o repo como uma coisa só.

O professor pediu **dois serviços implantáveis sozinhos**. Morando juntos, quem precisa aprender a separá-los é o pipeline.

O que o robô fez com o arquivo de vocês

participantes/
ruan.json
1 arquivo mudado



- | | |
|---------------|-------------|
| ● test mural | precisava |
| ● test api | desperdício |
| ● test shared | desperdício |

Os três ficaram **verdes**, e isso é o pior do caso: ninguém percebe o desperdício quando ele passa.

Path filter: ensina o robô a olhar o diff

Cada job ganha uma lista de pastas. Se nada daquela lista mudou no PR, o job não roda — e o GitHub o marca como **skipped**, não como falha.

A regra que importa

Quem mexe no **shared/** acorda todos. É o preço do código compartilhado, e está certo que seja assim.

```
on:
  pull_request:
    paths:
      - 'apps/mural/**'
      - 'participantes/**'
      - 'packages/shared/**'
```

mudou json mural roda • api e shared pulam
mudou
shared os três rodam

O CONSERTO N&ORDM; 2

Cache: pare de baixar a internet duas vezes

A máquina do GitHub nasce limpa e morre no fim do job. Sem cache, ela baixa as mesmas 600 dependências em cada PR, para sempre.

Uma linha resolve. Não é a maior economia do CI — a maior é o filtro, que nem abre o job. Mas é a mais barata.

```
- uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'
```

NPM INSTALL	TEMPO
sem cache	25 s
com cache	11 s

medido · 603 pacotes · 379 MB · runner ubuntu-latest

● AO VIVO – TROCAR PARA O NAVEGADOR

Dois commits, dois resultados

toco participantes/ 1 job roda, 2 ficam **skipped**

toco packages/shared/ os **3** acordam

O MESMO REPO, AGORA CONSERTADO

Vocês esperaram. Agora assistam.

o pipeline ingênuo levou _____ nos PRs de vocês

12 suítes contra 1

No mesmo PR, que mexe **num arquivo só**.

O ingênuo roda 3 pacotes × 4 versões de Node.

O rápido roda o pacote que mudou.

Mudou no código: **5 linhas de YAML**. Nenhuma linha de teste foi apagada.

E O RELÓGIO? DEPENDE DO TAMANHO DO PROJETO

NODE_MODULES	INGENUO	RAPIDO
vazio este repo, de brinquedo	47 s	30 s
83 MB 151 pacotes	58 s	27 s
379 MB 603 pacotes — app de verdade	2m 37s	40 s

O ingênuo **cresce com o projeto**. O rápido quase não se move.

medido · mesmo commit, mesmos testes ·
docs/aprofundamento/medicao.md

E AÍ ENTRA O CD

Ninguém da nossa equipe aprovou o PR de vocês.

O robô aprovou.

Branch protection

A main recusa qualquer PR cujo check não esteja verde. Nem o dono do repo passa por cima.

Auto-merge

Ficou verde, entra sozinho. Não fica esperando alguém acordar e clicar.

Deploy no merge

Entrou na main, vai pro ar. Sem ninguém subir arquivo à mão.

● AO VIVO — ABRAM O LINK DE NOVO

Vocês estão no ar.

Dêem F5 na página. O nome de quem ficou verde já está lá — e nenhum de nós quatro subiu nada.

PR verde → merge automático → build → **página no ar**

tempo total: menos de um minuto • ninguém revisou, ninguém subiu arquivo

● AO VIVO – UM ÚLTIMO PR

Este está verde. A gente merge?

✓ test mural	passou
✓ test api	skipped
✓ lint	passou

**Mão pra cima quem
mergearia.**

Agora mão pra cima quem não mergearia.

O LIMITE DA FERRAMENTA

Verde não significa “está certo”.

Significa “o que a gente ensinou o robô a conferir, passou”.

O pipeline cobre o que **tem teste**.
O resto ele nem olha.

Por isso **code review existe**
mesmo com CI verde.

Teste novo entra **junto** com o
código, no mesmo PR.

Três coisas para levar pra próxima quinzena

1 Põe **paths** no workflow hoje

São quatro linhas e vocês vão ter dois serviços no mesmo repo até o Encontro 6.

2 Leia o **fim** do log, não o log

X vermelho → Details → step vermelho → rolar até o fim → a linha **Error:**.

3 Verde é o que vocês **ensinaram** ele a conferir

Nada mais. Teste novo entra no mesmo PR do código novo.

O material está no repositório

[docs/aprofundamento/](#)

Os dois workflows completos, o passo a passo do path filter job por job, e o repositório da demo para vocês abrirem.

EQUIPE

Gilmar Silva · Julia Ribeiro · Ruan Pablo · Samuel Vanoni

Obrigado — e perguntas?



[github.com/SamuVanoni/
GCC267-cicd-monorepo](https://github.com/SamuVanoni/GCC267-cicd-monorepo)
→ [docs/aprofundamento](#)